

Certified Evaluation of Autonomous Cyber Defenders: A Five-Step Delay Breaks the CAGE-2 Champion

Anonymous Author(s)
Anonymized for double-blind review

Abstract—Autonomous cyber defenders, from reinforcement-learning (RL) agents to language-model agents, are compared by their mean reward over a fixed number of episodes against the scripted attackers they were built for. Such scores carry no guarantee and say nothing about attackers outside that set. We propose a *certified evaluation* protocol: distribution-free confidence bounds on a defender’s reward and on the probability that the mission-critical server is breached, valid simultaneously at every number of episodes (so evaluation may stop as soon as a verdict is reached), valid against an attacker that adapts across episodes, and combined into a joint worst case over a set of attackers that includes ones the defender has not seen. Applied to CAGE Challenge 2, the protocol shows that the challenge’s winning RL agent, which is never breached by the two official attackers (certified breach rate at most 0.04), is breached in 64% of episodes when the official B_line attacker simply waits one to five steps before starting: its attacker fingerprint misfires and it falls back to doing nothing. An adaptive attacker finds this on its own. The winner ranks first of 5 defenders by mean reward and 4th by certified worst case, and a one-line change, which the certificate pointed to, restores a certified breach rate of at most 0.04 on every attacker. A language-model attacker (Claude Haiku 4.5), told only to find what breaches the defender, discovers the same blind spot unaided: against the winner it breaches 0.68 of episodes — matching the best fixed strategy in hindsight (0.70) and far above uniform choice (0.17) — yet against the repaired winner it breaches 0.00. Given instead CybORG’s raw actions, a language model operates the full kill chain unaided but, attacking at once, rarely rediscovers the timing weakness the selector sought out; the blind spot holds across a whole delay family, and the protocol carries over to a second environment (CAGE Challenge 1). Language-model defenders do not share the winner’s blind spot: Claude Sonnet 5 was breached in 0 of 34 episodes and Claude Haiku 4.5 in 14% of delayed-attacker episodes. Certifying them to the winner’s precision, however, would take hundreds of dollars and tens of hours of evaluation per model; we quantify that cost.

Index Terms—autonomous cyber defense, CybORG, CAGE challenge, LLM agents, reinforcement learning, evaluation, anytime-valid inference, robustness

1. Introduction

Autonomous defense agents are now trained and benchmarked in cyber-operations simulators [1], and language-model (LLM) agents have joined the reinforcement-learning (RL) agents that dominate the leaderboards [2]. The CAGE challenges rank submissions by mean reward against the same rule-based attackers used during development, which rewards solutions fitted to known adversaries [3]. Reported scores are means and standard deviations over a fixed number of episodes; they carry no guarantee, and they are silent about what an attacker outside the evaluation set would do.

We ask what a defender can be *certified* to achieve. Our protocol gives, for each defender and attacker, confidence bounds on the expected episode reward and on the probability that the operational server is breached that are distribution-free (they use only the known range of the outcome), valid simultaneously at every number of episodes (the evaluator may look after every episode and stop as soon as the answer is settled), and, for an attacker that adapts across episodes, valid for the running average of what the attacker achieves. A defender’s certified worst case is the least favourable bound over a set of attackers that deliberately includes ones it was not built against.

Contributions. (i) A certified evaluation protocol for autonomous defenders with time-uniform, distribution-free bounds, a martingale variant for adaptive attackers, and a joint worst case over attackers (Section 3). (ii) A faithful, framework-free port of the CAGE Challenge 2 winning agent and two unseen attackers that only re-sequence the official attackers’ actions (Section 4). (iii) The finding that the winner’s guarantees do not transfer: a delay of one to five steps defeats it, an adaptive attacker discovers this unaided, the certified ranking reverses, and a one-line repair restores the guarantee (Section 5). (iv) Claude defenders evaluated under the same protocol, with and without extended thinking, and the episode and dollar cost of certifying an LLM defender. (v) A language-model attacker that, given only the goal of breaching the defender, autonomously finds and exploits the winner’s blind spot yet is defeated by the one-line repair; and a primitive-action variant that issues raw CybORG actions and operates the full kill chain unaided, separating operating an attack from discovering the timing weakness. (vi) Evidence the finding is robust: the blind spot holds across a parameterized delay family and the protocol

carries over to a second environment (CAGE Challenge 1). Code, data and transcripts will be released.

2. Background and related work

CybORG [1] is a cyber-operations gym; CAGE Challenge 2 (Scenario 2) defends an enterprise and operational network against the scripted attackers B_line, which heads directly for the operational server, and Meander, which explores the network first. The winning entry [4] combines PPO policies trained against each attacker with greedy decoy placement and selects a policy by fingerprinting the attacker from early scans. Later work studies optimal CAGE-2 strategies with causal models and tree search [5], generalisation of graph-based RL defenders to new topologies and attackers [3], empirical game-theoretic evaluation of autonomous defenders [6], and LLM defenders in CybORG’s CAGE-4 scenario, which perform comparably to RL agents while being slower [2]. These studies report empirical means; none certifies a defender or reports a worst case with guarantees. Our bounds build on betting confidence sequences [7] and time-uniform martingale bounds [8].

3. Certified evaluation

An episode against attacker a yields bounded outcomes: the reward $R \in [r_{\min}, 0]$ with $r_{\min} = -474$ for a 30-step Scenario-2 episode (at most 15.8 lost per step: every host fully compromised, the operational server impacted and a restore paid for), and the breach indicator $B \in \{0, 1\}$ (the attacker successfully impacted the operational server at least once).

Fixed attacker. Episodes are i.i.d. For outcomes rescaled to $[0, 1]$ and each betting fraction λ on a fixed grid, $K_t(m) = \prod_{s \leq t} (1 + \lambda(y_s - m))$ is a non-negative martingale when the mean equals m ; their average over the grid is too. By Ville’s inequality, $\{m : K_t(m) < 1/\alpha\}$ is a confidence set valid for all t at once with probability $1 - \alpha$ [7]. We report two-sided bounds at $\alpha = 0.05$.

Adaptive attacker. If the attacker’s choice for episode t depends on past outcomes, the target is the running average $\bar{\mu}_t$ of the conditional means. With increments in $[0, 1]$, $\sum_{s \leq t} (y_s - \mu_s)$ is a martingale with $1/4$ -sub-Gaussian increments, and the normal-mixture boundary $\sqrt{2(t/4 + \rho) \log(\sqrt{(t/4 + \rho)/\rho}/\alpha)}$ holds for all t [8].

Worst case and stopping. The certified worst case over attackers A is the least favourable bound with $\alpha/|A|$ per attacker, so it holds jointly. Because all bounds are time-uniform, an evaluator may stop at the first episode at which a bound settles a question (for example, whether the breach rate is below 0.2) without invalidating the result. Simulation tests confirm the stated coverage under per-episode peeking and under an adaptive adversary.

4. Setup

Environment. CybORG Scenario 2 at the official CAGE-2 commit, 30-step episodes, seeded so that every

(defender, attacker, seed) triple is reproducible and episodes are paired across defenders. We also evaluate the generic defenders on CAGE Challenge 1 (Scenario1b) as a second environment.

Attackers. The official B_line and Meander (*seen*); two *unseen* re-sequencings of their own actions: *delayed B_line*, which sleeps for a uniformly random one to five steps first, and *Meander*→*B_line*, which explores like Meander for four steps and then follows B_line; and an *adaptive* attacker that picks among these four before each episode by EXP3 [9], rewarded by the damage it did. A fixed-delay family sweeps the pause deterministically from one to 10 steps. For the *primitive-action* attacker we instead drive Red externally through CybORG’s six native actions (discover systems, discover services, exploit, escalate privilege, impact, sleep), the target of each restricted to what the attacker has discovered; the defender keeps its own interface, and a scripted policy over this interface reaches and impacts the server, confirming the interface is faithful.

Defenders. Sleep (no action); the official React-remove and React-restore agents; the challenge winner [4], ported to NumPy from its released checkpoints (a restricted unpickler reads the weights without PyTorch), which reproduces its published Meander score (-5.58 against -5.54 over 1000 episodes); against B_line it scores -5.24 against the published -3.41, consistent with an unpublished change to CybORG’s host model that the authors describe; we evaluate the agent on the unmodified official environment. *Winner + fallback* differs in one line: when the fingerprint matches neither known attacker it loads the B_line policy instead of sleeping.

LLM defenders. Each step, Claude receives the step number, CybORG’s Blue table (per host: subnet, activity seen this step, estimated compromise) and its last five actions with their outcomes, and replies with one JSON action (Sleep, Monitor, Analyse, Remove, Restore or Decoy on a host); invalid replies become Sleep and are counted. Calls are stateless and made through the Claude Code command-line interface with no tools, no settings and an empty working directory. We evaluate Claude Haiku 4.5 and Claude Sonnet 5 without extended thinking and Haiku 4.5 with it, on the same seeds as the other defenders.

5. Results

The winner’s guarantee does not transfer. Table 1 and Figure 1. Against the attackers it was built for, the challenge winner is never breached in 200 episodes each (certified breach rate at most 0.04; mean reward -5.6). Against delayed B_line it is breached in 64% of episodes (certified interval 0.51–0.74; mean reward -91.0): the delay changes the scan pattern its fingerprint expects, it classifies the attacker as neither, and it falls back to its sleep policy. Meander→B_line does not fool it. The adaptive attacker discovers the weakness without being told: it chose the delayed strategy in 79 of 200 episodes and breached the winner in 24% (certified running-average bound 0.36). Figure 2 shows the certificates

as episodes accumulate: the failure is certified (breach rate above 0.2) after a few dozen episodes.

Rankings reverse. Table 2. By mean reward on the seen attackers, the winner ranks 1 of 5; by certified worst case over all four fixed attackers it ranks 4, behind React-restore (worst-case breach bound 0.22) and even React-remove (0.71), with a bound of 0.76.

The certificate points to a one-line repair. Because the worst case names the attacker and the episodes, the failure is easy to localise. Loading the B_line policy when the fingerprint is ambiguous gives a certified breach rate of at most 0.04 on every fixed attacker (mean reward -4.3 against delayed B_line) and at most 0.13 against the adaptive attacker.

The blind spot is a threshold, not a lucky delay. Sweeping a fixed delay from one to 10 steps (100 episodes each, Figure 4) shows the weakness is not an artefact of the one-to-five-step choice. A one-step delay does not trigger it (breach 0.00, certified at most 0.07); from two steps on the fingerprint misfires and the winner is breached in 0.80 of episodes (certified at least 0.64), and in 0.55–0.80 across every delay up to 10 (certified at least 0.38 throughout). Delaying the *explorer* instead does not fool it (0.05, certified at most 0.16), so the weakness is specific to a delayed *direct* attack. The one-line repair holds across the whole family (breach at most 0.07 at every delay), as does React-restore (0.12).

LLM defenders. Claude Haiku 4.5 without extended thinking (27 episodes per attacker; \$0.07 and 1.6 minutes per episode) was breached in 11% of B_line episodes and 14% of delayed-B_line episodes, and never by Meander or Meander→B_line. Unlike the winner it does not act on a guess of which attacker it faces, and the delay did not raise its breach rate. On the 28 shared seeds against delayed B_line, the winner alone was breached in 16 and Haiku alone in 1; a conventional fixed-sample sign test gives $p = 0.000$, but at this sample size the time-uniform bound does not yet certify the difference. Claude Sonnet 5 without thinking (7–10 episodes per attacker, \$0.32 per episode) was breached in 0 of 34 episodes, against every attacker including the delayed one, with a mean reward of -9.4 on the seen attackers (winner -5.6, React-restore -20.6). On the 10 seeds it shared with the winner against delayed B_line, the winner alone was breached in 7 and Sonnet alone in 0 (conventional sign test $p = 0.016$). Extended thinking did not help Haiku: with it (5–5 episodes per attacker) each episode cost \$0.29 and took 9.5 minutes, and it was breached in 40% of B_line episodes. Invalid replies were rare (0 of 3300 calls for Haiku, 3 of 1020 for Sonnet). These results are descriptive: with so few episodes the certified worst-case breach bounds remain wide (Haiku 0.55, Sonnet 1.00). Certifying an LLM defender to the precision we certify the winner (200 episodes per attacker) would cost about \$52 and 22 hours with Haiku and \$253 and 36 hours with Sonnet through this interface; the whole LLM study cost \$23.74.

An LLM attacker finds the blind spot. We turn the language model on the offensive side. Each episode it chooses one of the four attack strategies to launch against a fixed

defender, sees whether that breached the operational server, and adapts – the role the scripted EXP3 adversary played, but with the model, not a bandit algorithm, deciding. It is told only to find and exploit whatever breaches the defender; the winner’s fingerprint weakness is never mentioned. Table 3 and Figure 3: against the winner, Claude Haiku 4.5 breaches 0.68 of 40 episodes (certified 0.39–0.93), choosing the delayed-start attack in 98% of episodes. That matches the best fixed strategy in hindsight (0.70), far exceeds uniform choice (0.17), and exceeds even the scripted EXP3 adversary (0.23), which keeps exploring where the model commits once it finds the weakness. Against the one-line-repaired winner the model breaches 0.00 despite trying the delayed attack, and against React-restore it settles on the direct attack and breaches 0.17 (best fixed 0.17). Claude Sonnet 5 leans the same way – delayed start in 57% of its 30 episodes against the winner – but less decisively, so it breaches only 0.30 (certified 0.00–0.65); at this episode count that interval does not clear uniform choice (0.17), where Haiku’s does. Neither model breaches the repaired winner (0.00 of 30). The whole attacker study cost \$1.20.

A primitive-action LLM attacker. The strategy selector chose among ready-made attacks; we also give a language model CybORG’s *raw* Red actions and have it operate itself, choosing one action and target per step from what it has discovered – scan a subnet, scan and exploit a host, escalate, pivot, and finally Impact the operational server (Section 4). As a gate, a scripted policy over the same interface breaches the undefended Sleep defender, so a failure to breach is the attacker’s doing, not the interface’s. Over 15 episodes each (results descriptive, with wide bounds at this size) Claude Haiku 4.5 drives the full kill chain unaided, breaching Sleep in 0.53 of episodes. Against the winner it breaches far less, 0.13, against the strategy selector’s 0.68: usually attacking at once, it mostly meets exactly the fingerprint the winner was built for, and only occasionally wanders into a pause long enough to defeat it, rather than seeking that pause out as the selector (with the delayed attack on its menu) does. The repaired winner and React-restore are never breached (0.00 and 0.00). This separates *operating* the kill chain, which the model does, from reliably *discovering* the timing weakness, which needs either the scripted delay or a menu containing it. The primitive study cost \$3.49.

The cost of certification. With time-uniform bounds the evaluator can stop early: for the non-LLM defenders, 18 of 20 breach questions (rate below or above 0.2) were settled after a median of 38 episodes (range 12–149), against the 200 we ran and the 1,000 used by the challenge. Settling a question takes few episodes when the breach rate is near 0 or far from the threshold and many when it is close, which matters for LLM defenders whose episodes cost money and minutes.

A second environment. To check the protocol is not specific to Scenario 2, we apply it unchanged to CAGE Challenge 1 (CybORG Scenario1b), a related network with the same operational-server objective, for the 3 generic defenders (the winner’s policy is specific to Scenario 2) over 200 episodes per attacker. It yields certified rankings

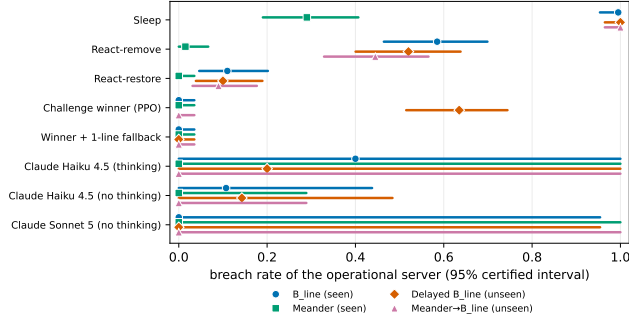


Figure 1. Breach rate of the operational server per defender and attacker, with the time-uniform 95% certified interval. Non-LLM defenders: 200 episodes per attacker; LLM defenders: fewer (Table 1), hence wider intervals.

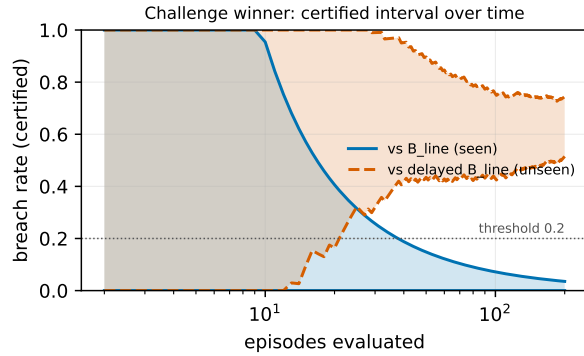


Figure 2. The challenge winner's certified breach-rate interval as episodes accumulate, against a seen attacker and against delayed B_line. Bounds hold at every episode count, so the evaluation may stop as soon as an interval clears the threshold.

there too: over all fixed attackers React-restore is certified to a breach rate of at most 0.22, ahead of React-remove (0.71) and Sleep (1.00). The ordering, and React-restore's bound, match its standing on Scenario 2, so the certified score is stable across the two environments rather than an artefact of one. The winner's blind spot is specific to Scenario 2's agent and so has no analogue here.

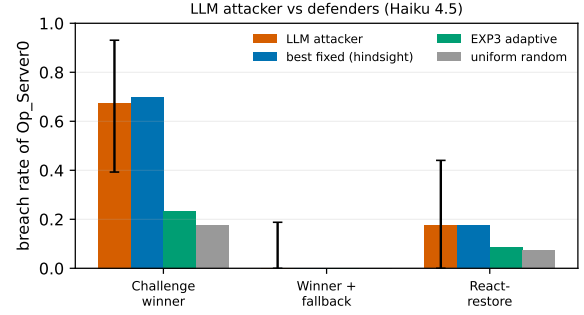


Figure 3. LLM attacker (Haiku 4.5) breach rate per defender, with its 95% certified interval, next to the best fixed strategy in hindsight, the scripted EXP3 adversary and uniform-random choice. The model matches the best fixed strategy against the winner and is defeated by the one-line repair.

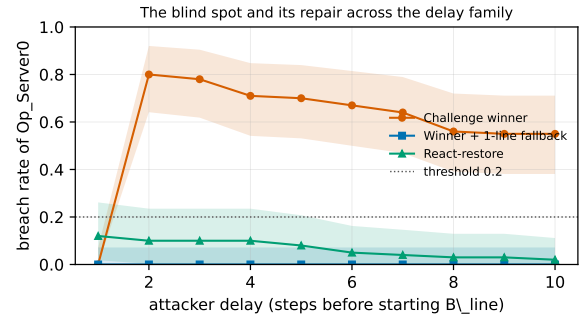


Figure 4. Breach rate of the operational server against a B_line that waits a fixed number of steps before starting, with 95% certified bands. The winner is safe at a one-step delay but breached from two steps on, across the whole family; the one-line repair and React-restore stay near zero.

TABLE 1. BREACH RATE OF THE OPERATIONAL SERVER: MEAN, AND IN PARENTHESES THE TIME-UNIFORM 95% CERTIFIED UPPER BOUND, PER DEFENDER AND ATTACKER; n EPISODES PER ATTACKER. THE ADAPTIVE ATTACKER’S BOUND IS FOR THE RUNNING AVERAGE OF ITS CONDITIONAL BREACH PROBABILITIES.

Defender	B_line	Meander	Delayed B_line	Meander→B_line	Adaptive	n
Sleep	0.99 (1.00)	0.29 (0.41)	1.00 (1.00)	1.00 (1.00)	0.93 (1.00)	200
React-remove	0.58 (0.70)	0.01 (0.07)	0.52 (0.64)	0.45 (0.57)	0.46 (0.58)	200
React-restore	0.11 (0.20)	0.00 (0.04)	0.10 (0.19)	0.09 (0.18)	0.09 (0.21)	200
Challenge winner	0.00 (0.04)	0.00 (0.04)	0.64 (0.74)	0.00 (0.04)	0.23 (0.36)	200
Winner + fallback	0.00 (0.04)	0.00 (0.04)	0.00 (0.04)	0.00 (0.04)	0.00 (0.13)	200
Haiku 4.5	0.11 (0.44)	0.00 (0.29)	0.14 (0.48)	0.00 (0.29)	–	27
Sonnet 5	0.00 (0.95)	0.00 (1.00)	0.00 (0.95)	0.00 (1.00)	–	7
Haiku 4.5 + thinking	0.40 (1.00)	0.00 (1.00)	0.20 (1.00)	0.00 (1.00)	–	5

TABLE 2. MEAN REWARD ON THE SEEN ATTACKERS AND ITS RANK, VERSUS THE CERTIFIED WORST-CASE BREACH BOUND OVER THE SEEN AND OVER ALL FIXED ATTACKERS AND ITS RANK (NON-LLM DEFENDERS, EQUAL SAMPLE SIZES), WITH THE ATTACKER ATTAINING THE WORST CASE.

Defender	Mean	Rk	Seen	All	Rk	Worst
Sleep	-128.8	5	1.00	1.00	5	B_line
React-remove	-56.4	4	0.71	0.71	3	B_line
React-restore	-20.6	3	0.21	0.22	2	B_line
Challenge winner	-5.6	1	0.04	0.76	4	Delayed
Winner + fallback	-5.6	1	0.04	0.04	1	B_line

TABLE 3. LLM ADAPTIVE ATTACKER: BREACH RATE OF THE OPERATIONAL SERVER (WITH THE TIME-UNIFORM 95% INTERVAL) AGAINST EACH DEFENDER, VERSUS THE BEST FIXED STRATEGY IN HINDSIGHT, UNIFORM-RANDOM CHOICE AND THE SCRIPTED EXP3 ADVERSARY, AND THE SHARE OF EPISODES THE MODEL CHOSE THE BEST STRATEGY.

Model	Defender	n	Breach (95%)	Best	Rand.	EXP3	Best-chosen
Haiku 4.5	Challenge winner	40	0.68 (0.39–0.93)	0.70	0.17	0.23	98%
Haiku 4.5	Winner + fallback	40	0.00 (0.00–0.19)	0.00	0.00	0.00	10%
Haiku 4.5	React-restore	40	0.17 (0.00–0.44)	0.17	0.07	0.09	88%
Sonnet 5	Challenge winner	30	0.30 (0.00–0.65)	0.67	0.17	0.23	57%
Sonnet 5	Winner + fallback	30	0.00 (0.00–0.26)	0.00	0.00	0.00	27%
Sonnet 5	React-restore	30	0.17 (0.00–0.49)	0.17	0.07	0.09	83%

6. Discussion

Leaderboards that score agents against the attackers they were tuned on can certify little about deployment: delaying an official attacker by one to five steps, using only its own actions, defeats the strongest CAGE-2 agent. Certified worst-case evaluation over a deliberately broadened attacker set catches this, names the failing attacker, and measures whether a fix works. The same lens applies to the offence: a language model told only to breach the defender rediscovers the winner’s blind spot on its own and exploits it harder than the scripted adaptive adversary, while the certified repair stops it – offence and defence judged on one scale. For LLM defenders the protocol makes a second point concrete: their per-episode cost makes the number of episodes the binding constraint, and time-uniform bounds let an evaluator spend only what a verdict requires.

7. Limitations

Simulation. CybORG Scenario 2 is a stylised network; results concern that environment. Our second environment, CAGE Challenge 1, shares CybORG’s engine and so is a related rather than independent test of generality; a genuinely different simulator is future work.

Worst case over a set. Certificates hold over the attackers we include, not all attackers; our unseen attackers are deliberately simple.

Port. The winner is evaluated through our NumPy port; it reproduces the published Meander score but not the B_line score, which we attribute to the authors’ unpublished environment change. The delayed-start failure does not rest on that score: it is driven by the winner’s own released attacker-selection rule – the delay makes its early-scan fingerprint match neither trained policy, so it loads the sleep policy and does nothing – a property of the published decision logic, not of our port’s absolute B_line reward.

LLM evaluation. Few episodes per attacker, one prompt, and the command-line interface’s default sampling; the LLM results are descriptive and their certified bounds are wide. Every language model we evaluate, as defender and as attacker, is a Claude model from a single vendor – the family that also assisted with this work’s code – so nothing here speaks to how models from other families behave.

Budget control. Parallel workers check a shared spending cap before each episode, so a run can exceed its cap by up to one episode per worker; the Sonnet run did.

Attacker scope. The two attackers test different things: the strategy selector supplies adaptation over ready-made attacks, and the primitive attacker operates the kill chain through CybORG’s action abstraction – still not raw network exploitation (no packet crafting or novel vulnerability discovery), which the simulator does not model.

Price of anytime validity. Time-uniform bounds are wider than fixed-sample tests at small n ; we report conventional paired tests alongside where relevant.

Artifact and AI use. Third-party code (CybORG, the winning agent) is fetched at pinned commits with hash-checked weights. All numbers are generated from committed tables; LLM transcripts and costs are recorded per episode. The analysis code and draft text were produced with the assistance of an AI system (Claude); results are reproduced by automated tests.

References

- [1] M. Standen, M. Lucas, D. Bowman, T. J. Richer, J. Kim, and D. Marriott. CybORG: A gym for the development of autonomous cyber agents. *arXiv:2108.09118*, 2021.
- [2] S. R. Castro, R. Campbell, N. Lau, O. Villalobos, J. Duan, and A. A. Cardenas. Large language models are autonomous cyber defenders. In *IEEE Conference on Artificial Intelligence (CAI)*, 2025. *arXiv:2505.04843*.
- [3] I. J. King, B. Bowman, and H. H. Huang. Automated cyber defense with generalizable graph-based reinforcement learning agents. *arXiv:2509.16151*, 2025.
- [4] J. Hannay. CAGE Challenge 2 winning agent: PPO with greedy decoys. Source code and report, 2022.
- [5] K. Hammar, N. Dhir, and R. Stadler. Optimal defender strategies for CAGE-2 using causal modeling and tree search. *arXiv:2407.11070*, 2024.
- [6] G. Palmer, L. Swaby, D. J. B. Harrold, M. Stewart, A. Hiles, C. Willis, I. Miles, and S. Farmer. An empirical game-theoretic analysis of autonomous cyber-defence agents. *arXiv:2501.19206*, 2025.
- [7] I. Waudby-Smith and A. Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B*, 2023.
- [8] S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 2021.
- [9] P. Auer, N. Cesa-Bianchi, Y. Freund, and R. E. Schapire. The non-stochastic multiarmed bandit problem. *SIAM Journal on Computing*, 2002.